

Test Summary Report

Clinic Booking System

Student Biruk Mulatu
Student ID [STUDENT ID — fill in before submission]
Instructor Abel Tadesse
Date 4 September 2026

1. What was tested

- All three domain rules (fee by age, booking eligibility, appointment lifecycle) and the Rule 3b late-cancellation guard, via 58 unit tests derived from equivalence partitioning, boundary value analysis, a decision table and state transition testing (full derivation in the test design document).
- The `AppointmentService` orchestration layer, against Mockito-mocked repositories, a `Clock` stub, and a mocked `NotificationService`.
- The same service wired to real Spring Data JPA repositories and a real H2 database (9 integration tests: 5 in `AppointmentServiceIT`, 4 in the MockMvc-driven `AppointmentJourneyIT`), including a Mockito spy on the real notification service.
- The full user journey — log in, view slots, book, see the fee on confirmation, cancel — driven through a real headless Chrome browser via Selenium and the Page Object pattern (2 system tests in `BookingJourneyIT`).
- Continuous integration itself: both the GitHub Actions workflow and a Jenkins pipeline (run via the provided Docker setup) were exercised end-to-end, not just reviewed as configuration.
- Regression detection: a deliberate one-line defect was introduced, observed failing in CI, and fixed (§3 of the defect log and metrics document).

Level	Test classes	Test methods
Unit	<code>FeeCalculatorTest</code> , <code>BookingPolicyTest</code> , <code>AppointmentStateMachineTest</code> , <code>AppointmentServiceTest</code>	58
Integration	<code>AppointmentServiceIT</code> , <code>AppointmentJourneyIT</code>	9
System	<code>BookingJourneyIT</code> (Page Objects: <code>LoginPage</code> , <code>SlotsPage</code> , <code>BookingPage</code> , <code>ConfirmationPage</code> , <code>MyAppointmentsPage</code> , plus a shared <code>AbstractPage</code>)	2
Total		69

58 : 9 : 2 is a genuine pyramid shape — the large majority of cases run fast and without a container or browser, with progressively fewer, slower, broader-scope tests above them.

2. What was not tested

- **Concurrency.** Two patients racing to book the same slot at the same instant is not tested; C1 (`existsBySlotAndStatusIn`) is correct for sequential requests but has no explicit test under concurrent load, and no database-level uniqueness constraint backs it up. This is the single largest functional gap and is carried as residual risk below.
- **Real SMS delivery.** By design (CLAUDE.md), `NotificationService` has only a logging implementation; there is no real gateway to test against.
- **Non-functional testing** — load, performance under many concurrent users, accessibility, and security penetration testing are all out of scope per the test plan (§1.2).
- **A handful of defensive/unreachable branches** — e.g. a catch-all path in `GlobalExceptionHandler` and a couple of simple getters never called by any test — are the reason whole-application instruction/line coverage (~90%/~89%) sits below the 100% branch figure on `core`; none of this uncovered code sits inside the three graded business rules.
- **The JSON API added under `backend/.../web/api`** for the separate React frontend (`frontend/`) is additive and untested — it delegates to the same `AppointmentService` the graded suite already covers, but the controllers and DTOs themselves have no dedicated tests of their own (see `README.md`).

3. Results against exit criteria

Against the exit criteria set in the test plan (§3.2):

Criterion	Result
<code>mvn clean verify</code> passes in one run (unit + integration + system)	Met — BUILD SUCCESS, 69/69 tests green
≥80% branch coverage on <code>et.aau.clinic.core</code> , gate enforced	Met — 100% (34/34 branches)
Every valid/invalid state transition in the 5×4 grid has an explicit test	Met — 5 valid tests + a generated, count-asserted set of all 15 invalid pairs
Every specified boundary value has an explicit test	Met — all of -1, 0, 17, 18, 64, 65, 120, 121 (fee); the 2h notice boundary; the 24h cancellation boundary
GitHub Actions green on <code>main</code> ; Jenkins run at least once via Docker	Met — both verified, Jenkins verified end-to-end with real build attempts, not just reviewed configuration
A deliberate regression introduced, caught by CI, then fixed, with evidence recorded	Met — see <code>docs/regression-demo.md</code> and defect log §3
All defects logged are Closed, or explicitly carried as residual risk	Met — DEF-001/002/003 all Closed; concurrency gap carried explicitly below

4. Outstanding defects and residual risk

Item	Status	Residual risk
DEF-001, DEF-002, DEF-003	All Closed, verified	None outstanding — each has a verification step confirming the fix under the same conditions that exposed it.
Concurrent double-booking of a slot	Not fixed — not attempted (out of scope per the test plan)	Medium. Low likelihood in a small clinic's real usage pattern (requests are not sub-second-frequent), but the current check-then-act pattern (<code>existsBySlotAndStatusIn</code> then <code>save</code>) is a textbook race condition. Would need either a unique database constraint on (slot, non-terminal-status) or a transactional lock to close.
Defect metrics based on a small sample (3 defects, 1 developer, no external review yet)	N/A	Low–Medium. DRE of 100% and 0 escaped defects reflect that no external party has evaluated the system yet, not that the system is defect-free; the instructor's own evaluation is the first real external validation this project will undergo (see §5's verification/validation distinction).

5. Release recommendation

Recommendation: ready to submit, with the concurrency gap disclosed as a known limitation rather than fixed.

All exit criteria in §3 are met, the coverage gate is exceeded (100% vs. an 80% target) on the graded package, all three defects found during development are closed and verified, and the regression-detection requirement has real, reproducible evidence (two CI run links, a failing one and a passing one) rather than a description of what would happen. For a course project whose explicit grading criterion is the testing effort rather than the application's production-readiness, this is a defensible stopping point.

The one open item — concurrent double-booking — is a real, named risk rather than an unknown one, which is itself the point of a test summary report: it exists precisely to say plainly what was not covered, not just to report a clean scorecard. If this were a real clinic system rather than a course project, that gap would block release; here, it is disclosed and left as a direction for further work rather than fixed under time pressure this close to the deadline, since fixing it correctly would mean adding a database constraint and its own test coverage rather than a quick patch.

6. Foundations reflection — error, fault, failure; verification vs. validation

The defect: DEF-003, the flaky Selenium page-object bug found while verifying the Jenkins pipeline (full write-up in the defect log).

`BookingJourneyIT` passed reliably in GitHub Actions but threw `NoSuchElementException` the first time it ran inside the Jenkins Docker container.

- Error** — the human mistake behind the defect was a mental model slip while writing the page objects: assuming that once Selenium's `click()` call returns after triggering a redirect, the destination page has already finished rendering. That assumption is not guaranteed by the WebDriver spec, which only promises that a triggered navigation has *started*, not that it has completed.
- Fault** — the resulting defect in the code: every page object called `driver.findElement(...)` directly, with no wait, immediately after a click that

caused a navigation. The fault sat dormant in every page object from Phase 7 onward.

- **Failure** — the observable, external symptom: `BookingJourneyIT` threw `NoSuchElementException` looking for `#confirmation-status`, but only under the tighter CPU budget of the Docker container — the same fault produced no failure at all on the faster GitHub-hosted runner, which is exactly why a fault can sit latent for a long time before a particular environment turns it into a visible failure.

Verification or validation? This was caught by **verification**, not validation. Verification asks "are we building the product right" — checking the system against its own specification and internal correctness, which is exactly what an automated system test running inside a CI pipeline does: it re-runs a known, specified user journey and checks the code behaves correctly, with no real user or stakeholder involved. Validation — checking with an actual user or against real-world use ("are we building the right product") — never entered into finding this defect at all; no person ever clicked through the failing journey. That the defect only appeared under one specific execution environment (constrained CPU) is itself a small lesson in why verification has to include more than one environment: the GitHub Actions run alone would have verified the same specification and still missed this fault entirely.